

1. But du TP:

L'objectif de ce TP est de concevoir divers programmes et projets intégrés, dans le but de s'initier aux systèmes embarqués. Pour cela, nous utiliserons la carte FOXG20 équipée d'un processeur ARM et fonctionnant sous Linux.

À la fin de ce TP, nous serons en mesure de :

- Comprendre l'architecture et le fonctionnement des GPIO sous Linux.
- Exécuter des commandes Linux sur la carte.
- Démarrer le noyau Linux sur la plateforme FOXG20 équipée d'un processeur ARM.
- Contrôler les ports d'entrées/sorties (I/O) de la carte.
- Mettre en place un serveur web embarqué.

2. La carte FOXG20

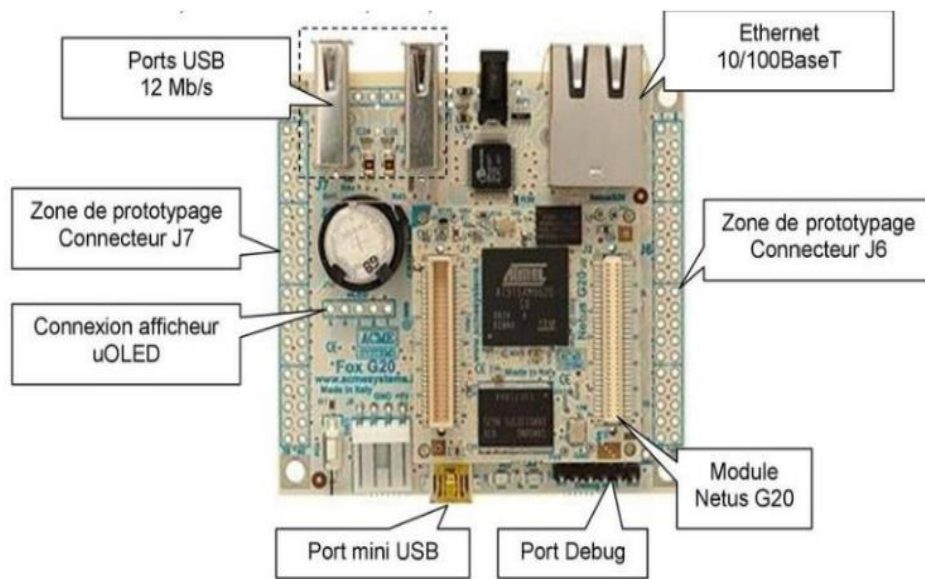
La carte FOXG20 est un système embarqué compact et économique, fonctionnant sous Linux et conçu autour du processeur ARM9™ AT91SAM9G20 cadencé à 400 MHz d'Atmel™. Ses caractéristiques la rendent idéale pour l'étude et le développement de mini-serveurs web, de proxys, de routeurs, de pare-feu, ainsi que d'autres systèmes de contrôle embarqués.

La carte est équipée de :

- Un connecteur d'alimentation,
- Un port Ethernet (Base 10/100),
- Deux ports USB 2.0 (hôtes),
- Un port client via mini-USB.

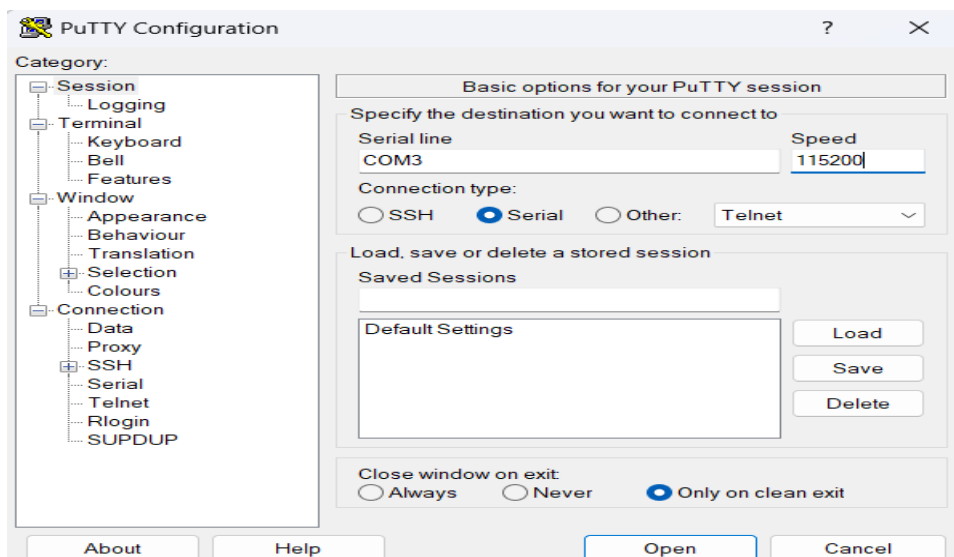
Elle dispose également de deux rangées de 20 trous, au pas de 2,54 mm, pouvant accueillir des connecteurs optionnels pour l'intégrer à une carte principale. Ces connecteurs offrent l'accès à de nombreux signaux :

- 28 ports d'entrées/sorties,
- 2 ports série (niveau 3,3 V),
- 4 entrées pour conversion analogique/numérique (A/N),
- Bus I2C™ / SPI™,
- Sortie PWM.



3.Accès à la carte FOX G20 :

Pour accéder à la FOX G20 via Putty sous Windows, il faut utiliser le port de débogage, qui permet d'établir une connexion USB. Cette opération nécessite l'insertion d'une carte microSD contenant l'OS Linux dans le lecteur situé à l'arrière de la carte.





Ensuite, connectez la carte au réseau à l'aide d'un câble LAN, puis branchez l'alimentation à la carte et au secteur

Pour relier les deux on utilise la structure suivante :

FTDI GND <> FOX GND

FTDI TX <> FOX RX

FTDI RX <> FOX TX

4.Utilisation des GPIO :

Les GPIO (General Purpose Input/Output) de la carte FOX G20 permettent de contrôler divers périphériques externes en configurant les broches en mode entrée ou sortie. Sous Linux, ces GPIO peuvent être manipulés via des commandes en ligne pour activer ou désactiver des ports, lire des signaux d'entrée, et gérer les périphériques connectés.

Chaque port GPIO de la carte FOX G20 est défini par :

- Le connecteur (par exemple, J6 ou J7),
- La broche (sa position sur la carte),
- L'identifiant de pin (pin ID),
- Le nom de la ligne (exemple : PA6).

Sous Linux, le système d'exploitation traite les ports GPIO comme des fichiers. Chaque port possède deux fichiers principaux :

- **value** : contient la valeur du port, soit 0 (état bas) soit 1 (état haut).
- **direction** : indique si le port est configuré en sortie (OUT) ou en entrée (IN).

Ces fichiers sont stockés dans un répertoire qui porte le nom du pin utilisé. Avant d'accéder aux fichiers, il est nécessaire d'activer (ou d'exporter) le pin à l'aide de la commande suivante :

```
echo 'pin ID' > /sys/class/gpio/export
```

Exemple

Pour activer et configurer la ligne 22 du connecteur J6, broche 37 :

1. Activer le pin

```
#echo 22 > /sys/class/gpio/export
```

2. Définir la ligne comme sortie :

```
#echo out > /sys/class/gpio/pioA22/direction
```

3. Définir la valeur à 1 pour allumer une LED connectée

```
#1 > /sys/class/gpio/pioA22/value
```

Paramètres de Connexion Série :

- **Port série :** COM3
- **Vitesse de Baud :** 115200

Procédure de Connexion :

- **Logiciel utilisé :** PuTTY
- **Login :** root
- **Mot de passe :** netusg20

Configuration Réseau :

- **Connexion Ethernet :** PC connecté au même réseau que la Fox G20 via un câble Ethernet pour faciliter la communication réseau.

Adresse IP :

- Pour obtenir l'adresse IP de la carte Fox G20, j'utilise la commande suivante :
`ifconfig eth0`

```
COM3 - PuTTY
RX packets:405 errors:48 dropped:0 overruns:0 frame:0
TX packets:48 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:42050 (41.0 KiB) TX bytes:9776 (9.5 KiB)
Interrupt:30 Base address:0x4000

root@netusg20:~# ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:04:25:aa:03:17
          inet6 addr: fe80::204:25ff:feaa:317/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:492 errors:48 dropped:0 overruns:0 frame:0
          TX packets:48 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:51679 (50.4 KiB) TX bytes:9776 (9.5 KiB)
          Interrupt:30 Base address:0x4000

root@netusg20:~# ifconfig eth0 192.168.2.104
root@netusg20:~# ifconfig eth0 192.168.2.104
root@netusg20:~# ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:04:25:aa:03:17
          inet addr:192.168.2.104 Bcast:192.168.2.255 Mask:255.255.255.0
          inet6 addr: fe80::204:25ff:feaa:317/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:844 errors:63 dropped:0 overruns:0 frame:0
          TX packets:63 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:116277 (113.5 KiB) TX bytes:12767 (12.4 KiB)
```

Figure :2 Adresse IP de la FOX G20

Comme on peut forcer une adresse précise par la commande **ifconfig eth0 192.168.2.104**, une fois l'adresse IP configurée, nous avons ouvert un navigateur web (Edge dans notre cas) et entré cette adresse dans la barre d'URL. Cela nous a redirigé vers une page web présentant deux environnements de travail : **Shell** et **Codiad**.

En sélectionnant **Codiad**, nous nous sommes connectés avec les identifiants suivants :

- **Login** : root
- **Mot de passe** : ariag25



Figure :3 Interface de programmation Codiad et accès sur la carte Fox G20

5. Les fonctions nécessaires prédéfini :

```
#include <string.h>

#include <stdio.h>

#include <stdlib.h>
```

```

#include <unistd.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <fcntl.h>
#include <time.h>
#define LOW 0
#define HIGH 1
#define IN 1
#define OUT 0
struct S_GPIO_LINE {
    char id_line[5];
    int id_pin;
    int direction;
    int value; };
typedef struct S_GPIO_LINE S_GPIO_LINE;
int tableau[4];
/* ***** */
/* LOAD_GPIO_LINE */
/* Initialisation of a GPIO line */
/* ***** */
int load_gpio_line(S_GPIO_LINE *ps_line )
{
    FILE *p_gpio_line;
    /* Exporting GPIO line */
    if ((p_gpio_line = fopen("/sys/class/gpio/export", "w")) ==
NULL)
    {
        printf("Cannot open export file.\n");
        exit(1);
    }
    fprintf(p_gpio_line,"%d",ps_line->id_pin);
    fclose(p_gpio_line);

```

```

        return 0;}

/* ***** */

/* SET_GPIO_DIRECTION */

/* Sets the direction (IN/OUT) of a GPIO line */

/* ***** */

int set_gpio_direction(S_GPIO_LINE *ps_line, int i_direction)
{
    int fd;//file

    char path[35];//chemin

    snprintf(path,35, "/sys/class/gpio/pio%s/direction",ps_line-
>id_line);

    printf("path:%s\n",path);

    fd = open(path, O_WRONLY);

    if (-1 == fd) {

        fprintf(stderr, "Failed to open gpio direction for writing!\n");

        exit(1); }

    if( i_direction ) {

        if (-1 == write(fd, "in", 36)) {

            fprintf(stderr, "Failed to set direction!\n");

            exit(1); }}

        else {

            if (-1 == write(fd, "out", 36)) {

                fprintf(stderr, "Failed to set direction!\n");

                exit(1);}}

            return 0;}

/* ***** */

/* SET_GPIO_LINE */

/* Sets the value of a GPIO line */

/* ***** */

int set_gpio_line(S_GPIO_LINE *ps_line, int value)
{
    FILE *p_gpio_value;

    char gpio_value[35];

```

```

    char c_value[2];

    if( 1) {

        sprintf(gpio_value, "/sys/class/gpio/pio%s/value",ps_line-
>id_line);

        /* Setting value */

        if ((p_gpio_value = fopen(gpio_value, "w")) == NULL)

        {   printf("Cannot open value file.\n");

            exit(1);   }

        rewind(p_gpio_value);

        sprintf(c_value, "%d", value);

        ps_line->value = value;

        fwrite(&c_value, sizeof(char), 1, p_gpio_value);

        fclose(p_gpio_value);}

    else{

        printf("Wrong access.\n");

        exit(1);}

    return 0;}

/* ***** */

/* GET_GPIO_LINE */

/* Gets value of a GPIO line */

/* ***** */

int get_gpio_line(S_GPIO_LINE *ps_line)

{

    FILE *p_gpio_value;

    char gpio_value[35];

    int value = 0;

    if( 1 )

    {sprintf(gpio_value, "/sys/class/gpio/pio%s/value", ps_line-
>id_line);

        /* Getting value */

        if ((p_gpio_value = fopen(gpio_value, "r")) == NULL)

        {   printf("Cannot open value file.\n");

```



```

        exit(1);    }

    ps_line->value = fgetc (p_gpio_value)-48 ;

    fclose(p_gpio_value);    }

else

{ printf("Wrong access.\n");

    exit(1);    }

return value;}

void conversion(int n)

{    int q,r,i=3;

    q=n;

    while(q>0)

    {    r=q%2;

        q=q/2;

        tableau[i]=r;

        i--;    }}

int permutation(int x)

{    if(x==0)

        return 1;

        return 0;}

/* exemple allumer une led                                */

int main() {

    S_GPIO_LINE LED;

    LED.id_pin=32;

    sprintf(LED.id_line, "B0");

    load_gpio_line(&LED);

    set_gpio_direction(&LED, 0);

    while(1)

    {    set_gpio_line(&LED,1);

        sleep(1);

        set_gpio_line(&LED,0);

        sleep(1)

```

```

    } return 0;
}

```

6. Familiarisation avec les commandes GCC :

Exercice 1 : Compilation de base

1. Créer le fichier source

Créez un fichier nommé `hello.c` et ajoutez le code suivant :

```

#include <stdio.h>

int main() {

    printf("Hello, World!\n");

    return 0;

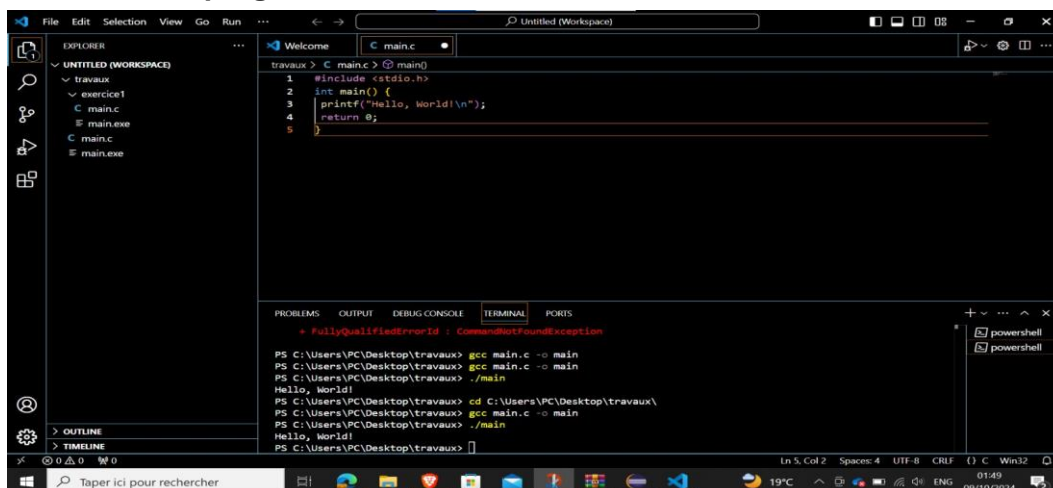
}

```

2. Compiler le fichier avec GCC:

```
gcc hello.c -o hello
```

3. Exécuter le programme `./hello`



Exercice 2 : Détection des erreurs de compilation

1. Créer le fichier source avec une erreur de syntaxe

Créez un fichier nommé `error_example.c` et ajoutez le code suivant :

```
#include <stdio.h>
int main() {
    printf("This will cause an error"
    return 0;
}
```

Remarque : Il manque une parenthèse fermante `)` à la fin de l'instruction `printf`.

2. Compiler le fichier et observer le message d'erreur

Dans le terminal, compilez le fichier avec GCC en utilisant la commande suivante :

```
gcc error_example.c -o error_example
```

3. Corriger l'erreur et recompiler

Modifiez le fichier `error_example.c` pour corriger l'erreur en ajoutant la parenthèse manquante

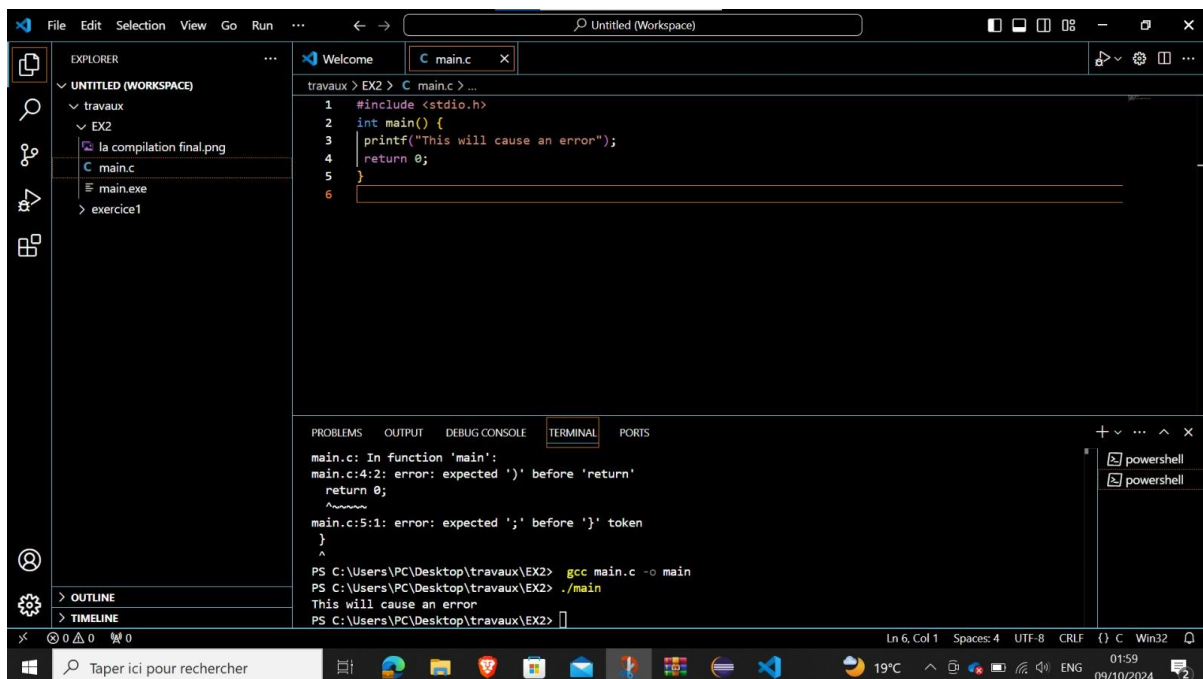
```
#include <stdio.h>
int main() {
    printf("This will cause an error\n");
    return 0;
}
```

4. Recompiler et exécuter le programme

```
gcc error_example.c -o error_example
```

Si la compilation réussit, exécutez le programme :

```
./error_example
```



Exercice 3 : Options du compilateur

Créer le fichier source

options_example.c

```
#include <stdio.h>
```

```
int main() {  
    int a = 5;  
    int b = 0;  
    int c = a / b; // Division par zéro  
    printf("Result: %d\n", c);  
    return 0;  
}
```

Compiler le fichier avec l'option -Wall

```
gcc -Wall options_example.c -o options_example
```

Exécuter le programme

`./options_example`

Résultat :

L'exécution du programme provoque un comportement indéfini:

Floating point exception (core dumped)

Cela se produit parce que la division par zéro est une opération illégale en C, entraînant une erreur d'exécution.

Exercice 4 : Utilisation de Makefile

1. Créer les fichiers source

Créez deux fichiers source : `main.c` et `functions.c` avec le contenu suivant.

main.c :

```
#include <stdio.h>
```

```
void greet()
```

```
int main() {
```

```
    greet();
```

```
    return 0;
```

```
}
```

functions.c :

```
#include <stdio.h>
```

```
void greet() {
```

```
    printf("Hello from functions.c!\n");
```

```
}
```

Créer un fichier Makefile

Dans le même répertoire que `main.c` et `functions.c`, on crée un fichier nommé `Makefile` et on ajoute le contenu suivant :

```
CC = gcc

CFLAGS = -Wall -g

TARGET = main_program

SOURCES = main.c functions.c

OBJECTS = $(SOURCES:.c=.o)

all: $(TARGET)

$(TARGET): $(OBJECTS)

$(CC) $(CFLAGS) -o $@ $^

%.o: %.c

$(CC) $(CFLAGS) -c $<

clean:

rm -f $(OBJECTS) $(TARGET)
```

Compiler le projet avec Make

Exécuter:

```
./main_program
```

Résultat:

Hello from functions.c!

1 : Fonction Allumé LED :

```
153     r=q%2;
154     q=q/2;
155     tableau[i]=r;
156     i--;
157 }
158
159 }
160 int permutation(int x)
161 {
162     if(x==0)
163         return 1;
164     return 0;
165 }
166
167 /* ***** */
168 /* exemple allumer une led */
169 /* ***** */
170 int main() {
171
172     S_GPIO_LINE LED;
173     LED.id_pin=32;
174     sprintf(LED.id_line, "B0");
175
176     load_gpio_line(&LED);
177     set_gpio_direction(&LED, 0);
178
179
180     while(1)
181     {
182
183
184         set_gpio_line(&LED,1);
185         sleep(1);
186         |
187         set_gpio_line(&LED,0);
188         sleep(1);
189     }
190
191 }
192 return 0;
193 }
194
```

2 : Fonction Allumé 3 LEDS :

```
157     }
158 }
159 }
160 int permutation(int x)
161 {
162     if(x==0)
163         return 1;
164     return 0;
165 }
166
167 /* ***** */
168 /* exemple allumer une led */
169 /* ***** */
170 int main() {
171
172     S_GPIO_LINE LED1;
173     LED1.id_pin=50;
174     sprintf(LED1.id_line, "B18");
175
176     S_GPIO_LINE LED2;
177     LED2.id_pin=33;
178     sprintf(LED2.id_line, "B1");
179
180     S_GPIO_LINE LED3;
181     LED3.id_pin=35;
182     sprintf(LED3.id_line, "B3");
183
184     load_gpio_line(&LED1);
185     set_gpio_direction(&LED1, 0);
186
187     load_gpio_line(&LED2);
188     set_gpio_direction(&LED2, 0);
189
190     load_gpio_line(&LED3);
191     set_gpio_direction(&LED3, 0);
192
193     while(1)
194     {
195
196
197
```

3 : Fonction Allumé LED par SWITCH :

```
mardigroupe2/troisled.c x mardigroupe2/allumeled.c x
156     } i--;
157 }
158 }
159 }
160 int permutation(int x)
161 {
162     if(x==0)
163         return 1;
164     return 0;
165 }
166
167 /* *****
168  * exemple allumer une led
169  * *****
170 int main() {
171     S_GPIO_LINE LED1;
172     LED1.id_pin = 48;
173
174     S_GPIO_LINE SWITCH;
175     SWITCH.id_pin = 35;
176
177     sprintf(LED1.id_line, "B16");
178     sprintf(SWITCH.id_line, "B3");
179
180     load_gpio_line(&LED1);
181     set_gpio_direction(&LED1, 0);
182
183     load_gpio_line(&SWITCH);
184     set_gpio_direction(&SWITCH, 1);
185
186     while (1) {
187         get_gpio_line(&SWITCH);
188         if (SWITCH.value == 1) {
189             set_gpio_line(&LED1, 1);
190         } else if (SWITCH.value == 0)
191         {
192             set_gpio_line(&LED1, 0);
193         }
194     }
195
196     return 0;
197 }
```

```
COM3 - PuTTY
troisled.c:53:5: note: expected 'struct S_GPIO_LINE **' but argument is of type 'S_GPIO_LINE'
troisled.c:184:5: error: incompatible type for argument 1 of 'set_gpio_direction'
troisled.c:53:5: note: expected 'struct S_GPIO_LINE **' but argument is of type 'S_GPIO_LINE'
root@netusg20:/var/Codiad/workspace/mardigroupe2# gcc -o troisled troisled.c
path:/sys/class/gpio/pioB16/direction
path:/sys/class/gpio/pioB1/direction
^C
root@netusg20:/var/Codiad/workspace/mardigroupe2# gcc -o troisled troisled.c
root@netusg20:/var/Codiad/workspace/mardigroupe2# ./troisled
path:/sys/class/gpio/pioB16/direction
path:/sys/class/gpio/pioB1/direction
^C
root@netusg20:/var/Codiad/workspace/mardigroupe2# gcc -o troisled troisled.c
root@netusg20:/var/Codiad/workspace/mardigroupe2# ./troisled
path:/sys/class/gpio/pioB16/direction
path:/sys/class/gpio/pioB1/direction
^C
root@netusg20:/var/Codiad/workspace/mardigroupe2# gcc -o troisled troisled.c
troisled.c: In function 'main':
troisled.c:188:9: error: expected ';' before 'if'
root@netusg20:/var/Codiad/workspace/mardigroupe2# gcc -o troisled troisled.c
root@netusg20:/var/Codiad/workspace/mardigroupe2# ./troisled
path:/sys/class/gpio/pioB16/direction
path:/sys/class/gpio/pioB1/direction
^C
root@netusg20:/var/Codiad/workspace/mardigroupe2# gcc -o troisled troisled.c
root@netusg20:/var/Codiad/workspace/mardigroupe2# ./troisled
path:/sys/class/gpio/pioB16/direction
path:/sys/class/gpio/pioB3/direction
^C
root@netusg20:/var/Codiad/workspace/mardigroupe2#
```